

HackIT

Олимпиада ИШИТР ТПУ по программированию

Россия, Томск, НТБ ТПУ

16 мая 2026 года

Региональный этап

Разбор задач основного тура

Предисловие



Автор задач: Никита Кузляев

Github: <https://github.com/NikitaKuzlyaeв>

Telegram: @kuzlyaeвnikita

Привет! Спасибо за интерес к олимпиаде!

Надеюсь, вам понравились задачи — это было сложно и долго: придумать, протестировать и собрать всё в единый тур. Это мой самый большой конкурс.

Если вы чувствуете, что можете сделать не хуже, у вас есть необычные идеи или вы просто хотите помочь в организации олимпиады — например, быть волонтером, участвовать в подготовке задач, или вы хотите предложить интересную активность — и при этом **вы учитесь в ИШИТР ТПУ**, напишите мне или организаторам.

P.S. Код решений задач доступен тут: <https://github.com/NikitaKuzlyaeв/HackIT>

Спасибо!!! :)

Задача А. Замки-картонки

Если можно вложить один замок в другой — давайте вложим, так как это не ухудшит ответ. Допустим, мы получили цепочки вложенности, в каждой из которых s_i замков. Если в цепочке k замков, то она дает k ответу

$$\frac{k \cdot (k - 1)}{2}.$$

Пусть есть две цепочки s_i и s_j такие, что $s_i \geq s_j$, и из s_j можно переложить один замок в s_i . Тогда две цепочки дают k ответу

$$\frac{s_i(s_i - 1)}{2} + \frac{s_j(s_j - 1)}{2}.$$

Но если мы переложим один замок из s_j в s_i , то получим

$$\frac{(s_i + 1)s_i}{2} + \frac{(s_j - 1)(s_j - 2)}{2}.$$

Ответ увеличится на

$$\left(\frac{(s_i + 1)s_i}{2} + \frac{(s_j - 1)(s_j - 2)}{2} \right) - \left(\frac{s_i(s_i - 1)}{2} + \frac{s_j(s_j - 1)}{2} \right) = s_i - s_j + 1.$$

Так как $s_i \geq s_j$, эта величина положительна. Значит, если есть возможность переложить замок из более короткой цепочки в более длинную, это выгодно.

Это означает, что нам нужно строить в первую очередь цепочки наибольшей длины.

Отсортируем массив. Одинаковые размеры не могут лежать в одной цепочке, поэтому если какое-то число встречается несколько раз, то его копии должны попасть в разные цепочки. При этом первую копию выгодно положить в самую длинную цепочку, вторую — во вторую по длине, и так далее.

Если идти по цепочкам явно и каждый раз искать подходящую, будет долго. Поэтому будем делать это лениво. Для каждой цепочки будем хранить только ее длину. После сортировки одинаковые числа идут подряд, значит, для текущего элемента легко понять, какой это по счету экземпляр среди равных. Пусть это pos -й экземпляр. Тогда кладем его в цепочку с номером pos .

Если такой цепочки еще не было, создаем новую цепочку длины 1. Иначе пусть ее текущая длина равна k . Новый замок добавляется в конец этой цепочки и образует пары со всеми k предыдущими замками, поэтому прибавляем k к ответу и увеличиваем длину цепочки на 1.

Так как элементы обрабатываются в отсортированном порядке, в каждой цепочке размеры строго возрастают. А одинаковые размеры попадают в разные цепочки, потому что имеют разные номера pos .

Сложность решения: $O(n \log n)$

Задача В. Портал в другой мир

Необходимо вывести значение

$$\left\lceil \frac{n}{a} \right\rceil + \left\lceil \frac{m}{b} \right\rceil$$

Сложность решения: $O(1)$

Замечания.

- При работе с целыми положительными числами можно воспользоваться таким трюком для деления с округлением вверх:

$$\lceil \frac{X}{Y} \rceil = \lfloor \frac{X + Y - 1}{Y} \rfloor$$

Тогда формула эквивалентна:

$$\lceil \frac{n}{a} \rceil + \lceil \frac{m}{b} \rceil = \lfloor \frac{n + a - 1}{a} \rfloor + \lfloor \frac{m + b - 1}{b} \rfloor$$

Это удобно, поскольку обычно целочисленное деление в языках программирования с округлением вниз. Также это надежнее и быстрее — у вас никогда не возникнут ошибки из-за неверного округления, так как вы не уходите в область чисел с плавающей точкой.

Пример верного кода:

```
n, m = map(int, input().split())
a, b = map(int, input().split())
res = (n + a - 1) // a + (m + b - 1) // b
print(res)
```

Пример кода, в котором потенциально что-то может пойти не так из-за работы с нецелыми числами:

```
import math
n, m = map(int, input().split())
a, b = map(int, input().split())
res = math.ceil(n/a) + math.ceil(m/b)
print(res)
```

Задача С. Другой мир

Применим идею динамического программирования. Пусть $dp_{i,j,t}$ — максимальное число вещей, которые мы можем собрать, рассмотрев первые i вещей, используя не более j коробок, если последняя выбранная вещь имеет тип t .

Будем считать, что зубная щетка имеет тип 0, а игрушка — тип 1.

Изначально все значения равны 0: мы можем не взять ни одной вещи.

Допустим, мы смотрим на вещь с номером $i + 1$, пусть ее тип равен $p \in \{0, 1\}$.

У нас есть два варианта.

- Первый вариант — пропустить эту вещь. Тогда состояние не изменится:

$$dp_{i+1,j,t} = \max(dp_{i+1,j,t}, dp_{i,j,t}).$$

- Второй вариант — взять эту вещь. Если до этого последняя выбранная вещь тоже имела тип p , то ее можно положить в текущую коробку, и новая коробка не нужна:

$$dp_{i+1,j,p} = \max(dp_{i+1,j,p}, dp_{i,j,p} + 1).$$

Если же до этого последняя выбранная вещь имела тип $1 - p$, то текущую коробку нужно закрыть и открыть новую коробку для типа p . Тогда количество использованных коробок увеличится на 1:

$$dp_{i+1,j,p} = \max(dp_{i+1,j,p}, dp_{i,j-1,1-p} + 1).$$

Таким образом, переход можно записать так:

$$dp_{i+1,j,p} = \max(dp_{i+1,j,p}, dp_{i,j,p} + 1, dp_{i,j-1,1-p} + 1).$$

Ответом будет максимум среди всех состояний:

$$\max_{0 \leq i \leq n+m, 0 \leq j \leq k, t \in \{0,1\}} dp_{i,j,t}.$$

В реализации можно хранить массив

$$dp[i][j][t].$$

Так как $n, m, k \leq 100$, полного трехмерного массива достаточно.

Сложность решения: $O((n + m) \cdot k)$

Задача D. Я показывал незнакомцам бинарный поиск

Ограничения на n и k достаточно небольшие, поэтому задачу можно решать итерационно: сделать k шагов и на каждом шаге выбирать такого пассажира, что, рассказав ему про бинарный поиск, мы получим максимальное удовольствие. Для этого нужно уметь быстро находить максимум и обновлять значение для выбранного пассажира.

К счастью, ничего придумывать не нужно — такая структура данных существует. Это *очередь с приоритетом*: вставка и извлечение максимального элемента работают за $O(\log size)$.

В начале удовольствие от первого рассказа каждому пассажиру равно a_i , поэтому для каждого пассажира положим в очередь пару (a_i, i) . Далее выполним k итераций. На каждой итерации достанем из очереди пару с максимальным первым значением, пусть это (x, i) . Тогда прибавим x к ответу, а затем положим этого же пассажира обратно в очередь уже со значением $(x - b_i, i)$, потому что после каждого рассказа удовольствие для него уменьшается на b_i .

После всех k итераций выведем полученный ответ.

Сложность решения: $O((n + k) \log n)$

Задача E. Прогулочки по лесу

В цикле от 1 до n будем переводить каждое число в римскую запись, затем в этой римской записи будем смотреть на цифры и прибавлять соответствующее число насечек к ответу.

Перевод любого числа от 1 до 3999 можно организовать «жадно»: в условии задачи была приведена таблица — в соответствии с ней любое число мы можем разбить на тысячи + сотни + десятки + единицы. Так, например, число 2567 мы представляем как $2567 = 2000 + 500 + 60 + 7$, а затем каждое число в представлении кодируем независимо: так, $2000 = \text{MM}$, $500 = \text{D}$, $60 = \text{LX}$, $7 = \text{VII}$, а само число представляется в виде конкатенации этих строк: $2567 = \text{MMDLXVII}$.

Второй шаг после перевода числа в римскую запись — посчитать из скольких насечек состоит эта запись. Это можно сделать проходом цикла по записи: используя условный оператор, определяем сколько нужно прибавить к ответу для очередного символа в записи (из скольких насечек состоит символ). Еще один вариант — заранее подготовить словарь для «мапинга» символа в число насечек — возможных реализаций много.

Сложность решения: $O(n)$

Замечания.

- По-хорошему, сложность перевода числа в римскую запись не бесплатна — она зависит от числа. Но у нас числа небольшие, и мы никогда не делаем больше 4 сравнений: по разряду тысяч, сотен, десятков и единиц — поэтому сложность считаем константой $C = 4$, а константы при оценке сложности опускаются.

Задача F. Огонь. Пламя. Гореть!

Будем перебирать город, в котором принимаем бой. Перебирать будем справа налево, то есть от города n к городу 1. Пусть сейчас рассматривается город i , а время прилета дракона в него равно T_i .

Сначала посчитаем лучников, которые могут прийти в город i слева, то есть из городов $1, 2, \dots, i$. Для этого построим префиксные суммы по массиву p :

$$pref_p[i] = p_1 + p_2 + \dots + p_{i-1}.$$

Тогда время пути лучника из города x в город i равно

$$pref_p[i] - pref_p[x].$$

Нужно найти самый левый город x , из которого лучники еще успевают прийти:

$$pref_p[i] - pref_p[x] \leq T_i.$$

Так как при уменьшении x расстояние только увеличивается, такой город можно найти бинарным поиском. После этого число лучников слева равно сумме

$$a_x + a_{x+1} + \dots + a_i,$$

которую можно быстро получить по префиксным суммам массива a .

Осталось научиться считать лучников справа, то есть из городов с номерами больше i . Будем идти справа налево и поддерживать очередь городов справа, лучники из которых еще могут дойти до текущего города. Также будем хранить сумму лучников в этих городах.

Пусть мы уже рассматриваем город i , а некоторый город $j > i$ находится справа. Лучники из города j должны успеть пройти расстояние от j до i до момента прилета дракона в i . Если это расстояние больше T_i , то такие лучники уже не могут помочь в городе i .

- Важное наблюдение: если лучники из некоторого города j справа уже не успевают дойти до города i , то лучники из любого еще более правого города тоже не успеют, потому что им нужно пройти еще большее расстояние. Значит, неподходящие города образуют префикс очереди, и их можно просто удалять из начала.

Именно поэтому можно хранить города справа в порядке добавления. Когда переходим к новому городу i , удаляем из начала очереди все города, лучники из которых уже не успевают дойти до i , и одновременно вычитаем их a_j из текущей суммы. После этого оставшаяся сумма в очереди — это ровно количество лучников справа, которые могут прийти в город i .

После подсчета ответа для города i добавим сам город i в очередь: при дальнейшем движении перебора влево он уже будет находиться справа от рассматриваемых городов.

Таким образом, для каждого города i мы считаем две величины: число лучников, которые могут прийти слева, и число лучников, которые могут прийти справа. Их сумма — количество воинов, которых можно собрать в городе i . Берем максимум по всем городам, а при равенстве оставляем город с наибольшим номером, так как перебираем города справа налево.

Корректность следует из того, что для каждого фиксированного города i мы отдельно учитываем все города слева и все города справа, из которых лучники успевают прийти к моменту прилета дракона. Слева подходящие города образуют суффикс, поэтому их граница находится бинарным поиском. Справа неподходящие города образуют префикс очереди: если более близкий к i правый город уже не успевает, то любой город еще правее тем более не успеет. Поэтому очередь всегда содержит ровно те правые города, лучники из которых могут помочь в текущем городе.

Каждый город добавляется в очередь один раз и удаляется из нее не более одного раза. Для каждого города дополнительно делается один бинарный поиск.

Сложность решения: $O(n \log n)$

Замечания.

- Бинарный поиск по самому левому городу, откуда могут прийти лучники в город i , для каждого очередного города i находит значение не большее значения, найденного на предыдущей итерации — соответственно, нам и не нужен бинарный поиск, а самый левый город мы можем поддерживать каким-нибудь указателем и постепенно его двигать влево. Это дает сложность решения $O(n)$.

Задача Г. Грубо? По-другому не умею. Прости, пожалуйста.

Переберем каждый символ строки s в цикле. Для каждого символа нужно определить, является ли он «цифровым» — для этого в языках программирования есть встроенные методы, так что, это делается легко. В крайнем случае можно было написать условный оператор на 10 условий (проверка на 0-9 + else). Если символ «цифровой», кастуем его к целочисленному типу и прибавляем к ответу.

Сложность решения: сложно.

Задача Н. Время идет, и мы не знаем, куда идти.

Зайдем в «305» и «309» кабинеты, заберем ключи, что находятся там. Далее, пока у нас есть ключи, будем выбирать среди ключей такой, что он от ранее непосещенного (закрытого) кабинета — пойдем в такой кабинет, возьмем все ключи там, и так далее. Если в какой-то момент среди наших ключей окажется «000», то мы победили, иначе, если среди наших ключей нет ни одного ключа от закрытого кабинета, то мы проиграли. Остается вопрос реализации.

Реализация такая: прочитаем все входные данные, и для каждого кабинета (первая запись в строке) запомним все ключи, что находятся внутри — хранить можно в структуре данных *словарь*, где ключом будет номер кабинета (строка), а значением список ключей внутри (список), тогда по номеру кабинета мы сможем быстро определять списки ключей — это очень удобно и пригодится нам далее.

В описанной ранее стратегии мы оперировали понятиями «посещенный» / «непосещенный» кабинет, и по номеру кабинета нужно уметь определять какой он — подойдет структура данных *сет*: создадим пустой сет и каждый раз, когда мы посещаем ранее непосещенный кабинет, будем добавлять его номер в сет, то есть в сете будут номера посещенных кабинетов. Очевидно, что два и более раз посещать один кабинет нет смысла — мы забираем все ключи из кабинета, когда оказываемся в нем впервые. Но тогда проверка *стоит ли нам идти в кабинет* будет обращаться к этому сету.

Осталось понять как можно поддерживать список кабинетов, в которые нужно сходить. Заметим, что в целом, нам без разницы, в каком порядке посещать кабинеты, если у нас есть ключи, открывающие несколько кабинетов — важно рано или поздно их посетить, но не порядок. Тогда определим порядок сами — будем поддерживать стек (можно и очередь) номеров кабинетов, которые мы хотим посетить.

Создадим стек из двух кабинетов «305» и «309». Далее, пока стек не пустой:

- Берем номер кабинета на верхушке стека;
- Проверяем, есть ли он в словаре (мы не ходим в несуществующие кабинеты);
- Проверяем, есть ли кабинет в сете (мы не ходим повторно в посещенные кабинеты);
- Если кабинета нет в сете, мы не были в нем. Двигаемся дальше;
- Добавляем кабинет в сет (так мы запоминаем, что посетили его);
- Берем список кабинетов, от которых есть ключ в текущем кабинете (определяем по словарю), и для каждого кабинета добавляем его в стек.

Если в процессе мы встретим ключ «000», то можно сразу прерываться и идти к выходу (вывести «Yes» и завершить программу). Иначе, если стек опустел, а ключ не был найден, выводим «No».

Сложность решения: $O(n)$

Замечания.

- Так как сказано, что в каждом из n кабинетов не более 10 ключей, то считаем сложность $O(Cn)$, где константа сокращается и получается $O(n)$. А ведь 10^9 — тоже константа...

Задача I. Первокурсница!!!

Вычислим сумму всех p_i — пусть она равна s . Затем найдем уникальные номера экзаменов среди конспектов — это можно сделать с помощью структуры данных *сет* — в сете хранятся уникальные элементы. Уникальные номера экзаменов нам необходимы, потому что может быть несколько конспектов к одному и тому же экзамену, но даже если у нас есть не один конспект к экзамену i , а несколько, то мы все равно вычтем p_i из s только один раз. Пройдем по числам в сете из для каждого числа i из сета, вычтем из s значение p_{d_i} . В переменной s будет ответ.

Сложность решения: $O(n + k)$

Замечания.

- Вместо сета, в силу небольших ограничений на n , можно использовать булевый массив, отмечая по индексу в нем те номера экзаменов, к которым есть конспекты.

Задача J. KafkaLoggingServiceAspectBeanPostProcessor

Разобьем строку s на слова w_i так, как это описано в условии: каждое слово начинается с заглавной буквы, а после нее идет максимальное количество строчных букв. Для каждого слова нам важны только три величины: первая буква, последняя буква и длина слова. Все буквы будем рассматривать без учета регистра.

Будем решать задачу динамическим программированием. Пусть dp_j — максимальная длина корректного названия среди уже рассмотренных слов, если последняя буква последнего выбранного слова равна j .

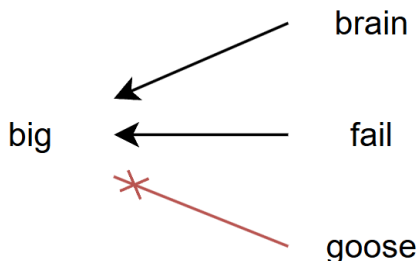
Здесь j — это буква от **a** до **z**. В реализации каждой букве сопоставляется число от 0 до 25.

Рассмотрим очередное слово w_i . Пусть оно начинается с буквы l , оканчивается буквой r , а его длина равна len .

Есть два варианта:

- не брать слово w_i — тогда все значения dp остаются прежними;
- взять слово w_i — тогда перед ним можно поставить любое уже выбранное название, которое заканчивается не на букву l .

Почему именно так: по условию запрещено, чтобы два соседних слова стояли рядом, если первое заканчивается на ту же букву, на которую начинается второе. Поэтому если текущее слово начинается с буквы l , то предыдущее выбранное слово не должно заканчиваться на l .



Значит, при добавлении слова w_i нужно найти

$$x = \max_{j \neq l} dp_j.$$

Тогда можно обновить состояние для последней буквы r :

$$dp_r = \max(dp_r, x + len).$$

Отдельно заметим, что слово можно взять и первым. Это уже учитывается формулой, потому что изначально все $dp_j = 0$, значит, x тоже может быть равен 0.

Важно обновлять dp_r аккуратно: при обработке одного слова нельзя сначала изменить dp_r , а потом использовать это новое значение для этого же слова. Поэтому для текущего слова сначала находим максимум по старому массиву dp , и только после этого делаем обновление.

Ответом будет

$$\max_{0 \leq j < 26} dp_j.$$

Сложность решения: $O(26 \cdot |s|)$

Замечание. Константа 26 в оценке. Ясно. Автор не разбирается в асимптотике...

Задача К. Не «лайки», а «сердца»

Очевидно, что полный перебор всех пар за $O(nm)$ не пройдет по времени, но нам хотелось бы уметь быстро определять для каждого элемента, например, из a , сколько для него подходящих элементов в b .

- Заметим, что если есть элементы a_i и b_j такие, что $a_i + b_j > k$, то для любого a_p такого, что $a_i \leq a_p$, так же выполняется $a_p + b_j > k$. Симметрично для b_j .

Замечание позволяет реализовать такую идею: перебирать элементы a_i первого списка и находить сколько чисел во втором списке таких, что $a_i + b_j > k$. Это можно сделать, отсортировав список b , а затем для каждого элемента a_i запускать бинарный поиск (lower bound).

Также возможен подход, использующий идею *двух указателей*:

- Отсортируем список a по неубыванию, а список b по невозрастанию;
- Будем идти слева направо указателем i по списку a , а по списку b двигать указатель j справа налево такой, что $a_i + b_j > k$, но $a_i + b_{j-1} \leq k$
- Тогда для элемента i подходят элемент j и все, что правее — то есть всего $m - j$ элементов.

Сложность решения: $O(n \cdot \log n + m \cdot \log m)$

Замечания.

- Существует решение с применением бинарного поиска за $O((n + m) \cdot \min(\log n, \log m))$.

Задача L. За окнами ночь

Заметим, что есть два типа коробок:

1. коробка i такая, что $a_i < b_i$;
2. коробка i такая, что $a_i \geq b_i$.

Коробки первого типа, если мы можем их купить, никак не портят ответ, а наоборот: улучшают — мы получаем $+1$ к ответу и при этом не уменьшаем число текущих сердец. Значит, такие коробки нужно покупать при первом возможном случае.

Второе замечание заключается в том, что если есть некоторый оптимальный порядок покупки коробок, максимизирующий ответ, то существует и порядок покупки, дающий такой же ответ, в котором сначала покупается некоторое количество коробок первого типа, а затем некоторое количество коробок второго типа. То есть после первой покупки коробки второго типа мы уже не покупаем коробки первого типа.

Мы не будем это доказывать в разборе, sorry. Но идея простая: коробки первого типа не уменьшают число сердец, поэтому их можно переносить раньше, и от этого возможность купить остальные коробки не ухудшится.

Тогда первая часть решения заключается в том, чтобы купить как можно больше коробок первого типа. Так как после покупки коробки первого типа наше число сердец не уменьшается, оптимальный порядок покупки — по неубыванию цены. Находим все коробки первого типа, сортируем по цене и идем по порядку, покупая коробки, пока это возможно.

Вторая часть решения заключается в том, чтобы выбрать как можно больше коробок второго типа. Пусть после первой части у нас осталось H сердец.

Для коробки второго типа выполнено $a_i \geq b_i$, поэтому после ее покупки число сердец уменьшается на

$$a_i - b_i.$$

Назовем эту величину потерей:

$$loss_i = a_i - b_i.$$

Пусть мы уже купили несколько коробок второго типа с суммарной потерей S . Тогда сейчас у нас осталось

$$H - S$$

сердец. Чтобы купить коробку i , должно выполняться

$$H - S \geq a_i.$$

Так как

$$a_i = b_i + loss_i,$$

получаем

$$H - S \geq b_i + loss_i,$$

то есть

$$S + loss_i \leq H - b_i.$$

В таких терминах это является классической задачей о расписании — есть n задач, каждая имеет продолжительность и дедлайн, нужно сделать как можно больше задач. *Мы не будем это доказывать в разборе, sorry.*

Каждая коробочка второго типа — это задача.

- Время выполнения задачи:

$$loss_i = a_i - b_i.$$

Именно столько сердец мы теряем, если покупаем эту коробочку.

- Дедлайн задачи:

$$H - b_i.$$

Неравенство

$$S + loss_i \leq H - b_i$$

означает следующее: если мы добавляем эту коробочку в выбранный набор, то суммарная потеря после ее покупки должна быть не больше ее дедлайна.

В классической задаче о расписании задачи рассматриваются в порядке возрастания дедлайна. У нас дедлайн равен

$$H - b_i.$$

Чем больше b_i , тем меньше дедлайн. Поэтому коробочки второго типа нужно рассматривать в порядке невозрастания b_i .

Дальше используем стандартный жадный алгоритм с кучей. Идем по коробочкам второго типа в порядке убывания b_i и поддерживаем выбранное множество коробок, а также их суммарную потерю $total_loss$.

Пусть мы рассматриваем коробочку i . Временно добавим ее в выбранные: увеличим суммарную потерю на $loss_i$ и положим $loss_i$ в кучу. Если после этого

$$total_loss > H - b_i,$$

то выбранный набор уже не укладывается в текущий дедлайн. Тогда нужно удалить из него коробочку с максимальной потерей.

Это оптимально: если мы хотим оставить как можно больше коробок и при этом уменьшить суммарную потерю, выгоднее всего удалить коробочку с максимальным $loss$. Для этого и нужна куча.

После обработки всех коробок второго типа в куче останется максимальное количество коробок второго типа, которые можно купить.

Итоговый ответ равен количеству купленных коробок первого типа плюс размер кучи после второй части.

Сложность решения: $O(n \log n)$

Замечания.

- Разбор больше задачи — база.

Задача М. The sun goes down.

Так как угольков всего три, можно просто перебрать все возможные варианты их взаимного расположения.

На самом деле вариантов всего два:

1. все три уголька лежат в один ряд;
2. сначала два уголька соединяются в один прямоугольник, а потом к нему присоединяется третий.

Будем использовать функцию, которая пытается соединить два прямоугольника. Два прямоугольника можно соединить, если после, возможно, поворота у них совпадает одна из сторон. Например, если их высоты равны, то их можно поставить рядом и получить новый прямоугольник, ширина которого равна сумме ширины.

В решении для каждой пары прямоугольников перебираются оба варианта поворота каждого из них. Если после поворота первые стороны совпали, то считаем, что прямоугольники можно соединить, и получаем новый прямоугольник:

$$(x_1, y_1) + (x_2, y_2) = (x_1, y_1 + y_2),$$

при условии, что

$$x_1 = x_2.$$

Теперь переберем порядок угольков. Пусть в каком-то порядке они равны

$$A, B, C.$$

Сначала попробуем соединить A и B всеми возможными способами. Для каждого получившегося прямоугольника попробуем соединить его с C .

Если хотя бы в одном переборе это удалось, значит, из трех угольков можно собрать один прямоугольник, и нужно вывести **Sorry**. Иначе вывести **Me**.

Сложность решения: $O(1)$

Задача N. Игроваренье

Одна комбинация всегда состоит ровно из двух действий: сначала парирование, потом пережат. Значит, на выполнение a комбинаций в любом случае нужно потратить $2a$ секунд.

Также каждая комбинация тратит $p + i$ очков выносливости. Поэтому всего на a комбинаций нужно $a \cdot (p + i)$ очков выносливости.

Изначально у игрока уже есть s очков выносливости. Если их хватает, то отдыхать вообще не нужно, и ответ равен просто $2a$. Иначе нужно восстановить недостающее количество выносливости:

$$a \cdot (p + i) - s.$$

Во время отдыха за одну секунду восстанавливается k очков, значит, на восстановление нужно

$$\frac{a \cdot (p + i) - s}{k}$$

секунд.

Итоговая формула:

$$2a + \frac{\max(0, a \cdot (p + i) - s)}{k}.$$

Сложность решения: $O(1)$

Задача О. — задача «ноль»? А почему она в конце?

Ограничения задачи достаточно небольшие, поэтому можно написать рекурсию (но с мемоизацией).

Пусть

$$f(n, m)$$

— количество способов раздать m косточек n пёселям так, чтобы каждый из них получил хотя бы одну косточку и никто не получил ровно 4.

Рассмотрим первого из оставшихся пёселей. Пусть он получил x косточек. Тогда должно выполняться:

$$x \geq 1, \quad x \neq 4.$$

Кроме того, оставшимся $n - 1$ пёселям нужно оставить хотя бы по одной косточке, поэтому

$$x \leq m - (n - 1).$$

После выбора x косточек для первого пёселя остается раздать

$$m - x$$

косточек между $n - 1$ пёселями. Значит, получаем переход:

$$f(n, m) = \sum_{\substack{1 \leq x \leq m - (n - 1) \\ x \neq 4}} f(n - 1, m - x).$$

Базовый случай: если остался один пёсель, то все оставшиеся косточки нужно отдать ему. Это можно сделать ровно одним способом, если количество косточек положительное и не равно 4:

$$f(1, m) = \begin{cases} 1, & m > 0 \text{ и } m \neq 4, \\ 0, & \text{иначе.} \end{cases}$$

Все значения будем считать по модулю

$$10^9 + 1000 - 7.$$

Чтобы не пересчитывать одинаковые состояния много раз, будем сохранять уже посчитанные значения $f(n, m)$ в словаре. Тогда каждое состояние вычисляется один раз.

На самом деле это та же идея динамического программирования: состояние задается парой (n, m) , а переход перебирает число косточек, которое получит текущий пёсель.

Рекурсия — это прикольно. Поэтому так...

Сложность такого решения: $O(n \cdot m^2)$